



Tecnológico Nacional de México

Instituto Tecnológico Superior de Felipe Carrillo Puerto

Ingeniería en Sistemas Computacionales



Asignatura

Programación BackEnd

Tema

2. API's

EVIDENCIA DE APRENDIZAJE

AA 2.1 API Reporte practico

Profesor

Ing. Paloma Gongora Sabido

Alumno (s):

Saul Alexander Che Caamal

(ISC- 8B)

Felipe Carrillo Puerto a 23 de marzo de 2026



2026
año de
**Margarita
Maza**





Interfaz de Programación de Aplicaciones

Las API, es la tecnología que potencia la comunicación entre el software en Internet.

- API son las siglas de Application Programming Interface (interfaz de programación de aplicaciones),
- Es un conjunto de normas y protocolos que definen cómo pueden interactuar distintos programas informáticos.
- Si el mundo entero funciona con software, es crucial pensar en la **comunicación** entre los diferentes programas que existen.

Ejemplo:

A partir de dos programas, diferentes Programa A (habla maya) y Programa B (habla inglés). Para que ambos interactúen, necesitan una interfaz que sirva de puente entre ellos.

Tipos de APIs

Existen diferentes tipos de API, como GraphQL, SOAP, REST, gRPC, etc.

Si se compara la creación de las APIs con la arquitectura de edificios; cada uno elige su estilo.

La API REST es la más importante para el desarrollo web. Una API RESTful sigue un conjunto específico de reglas, utilizando el protocolo HTTP para interactuar.



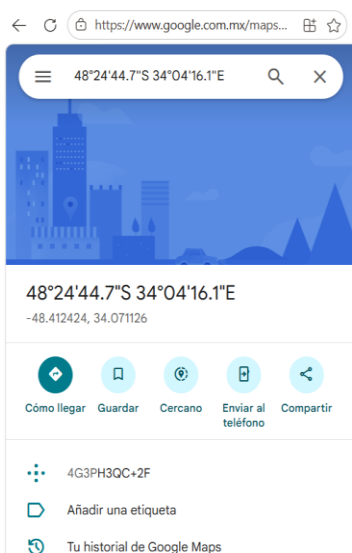


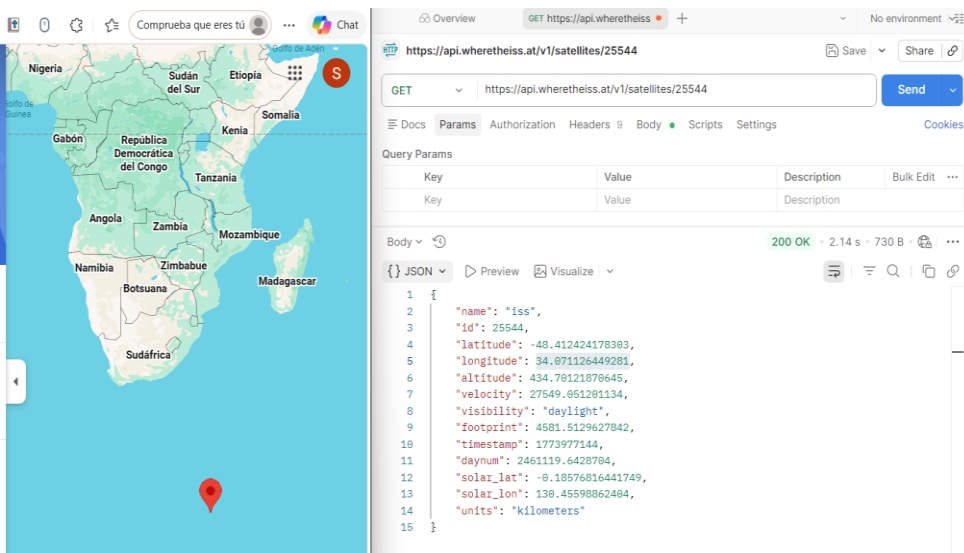
Petición a API

La [API REST de WTIA](#), es un servicio, para obtener la posición actual, pasada o futura de la ISS, información de la zona horaria de un conjunto de coordenadas y datos TLE de la ISS.

La Estación Espacial Internacional (ISS) es un satélite artificial habitable en órbita baja terrestre. Es un proyecto conjunto de cinco agencias espaciales participantes: la NASA estadounidense, la RSA rusa, la JAXA japonesa, la ESA europea y la CSA canadiense. La tripulación actual, compuesta por seis personas, forma parte de la Expedición 35, comandada por Chris Hadfield, de la CSA.

1. Solicitud GET para obtener latitud y longitud de la ISS.
 1. Usar Postman para hacer la solicitud HTTP (sin frontend o backend).
 2. Utilizar los valores latitud y longitud obtenidos para buscar la ubicación de la ISS en Google Maps.









Petición a API

JSON Placeholder es un servicio gratuito que proporciona datos ficticios para pruebas y prototipos. Puedes usarlo para obtener información sobre usuarios, publicaciones, comentarios, etc.

1. Utiliza Postman para realizar una solicitud GET a la siguiente URL para obtener una lista de usuarios: <https://jsonplaceholder.typicode.com/users>
2. La API devolverá un objeto JSON que contiene una lista de usuarios.

```
{ } JSON v Preview Visualize v
1  [
2      {
3          "id": 1,
4          "name": "Leanne Graham",
5          "username": "Bret",
6          "email": "Sincere@april.biz",
7          "address": {
8              "street": "Kulas Light",
9              "suite": "Apt. 556",
10             "city": "Gwenborough",
11             "zipcode": "92998-3874",
12             "geo": {
13                 "lat": "-37.3159",
14                 "lng": "81.1496"
15             }
16         },
17         "phone": "1-770-736-8031 x56442",
18         "website": "hildegard.org",
19         "company": {
20             "name": "Romaguera-Crona",
21             "catchPhrase": "Multi-layered client-server neural-net",
22             "bs": "harness real-time e-markets"
23         }
24     },
```

3. Puedes mostrar la información de los usuarios en tu aplicación o sitio web. Por ejemplo, podrías listar los nombres y correos electrónicos de los usuarios.
4. Para obtener otro tipo de recurso, puedes hacer una solicitud GET a:





Resources

JSONPlaceholder comes with a set of 6 common resources:

/posts	100 posts
/comments	500 comments
/albums	100 albums
/photos	5000 photos
/todos	200 todos
/users	10 users

<https://jsonplaceholder.typicode.com/posts>
Save
Share

GET
<https://jsonplaceholder.typicode.com/posts>
Send

Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body
Cookies
Headers 24
Test Results
200 OK • 94 ms • 7.98 KB

JSON
Preview
Visualize

```

1 [
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8   {

```



2026
año de
Margarita Maza





https://jsonplaceholder.typicode.com/comments/1

Save Share

GET https://jsonplaceholder.typicode.com/comments/1

Send

Docs Params Authorization Headers 9 Body Scripts Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 24 Test Results

200 OK 31 ms 1.27 KB

JSON Preview Visualize

```
1 {
2   "postId": 1,
3   "id": 1,
4   "name": "id labore ex et quam laborum",
5   "email": "Eliseo@gardner.biz",
6   "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
7 }
```

Overview

GET https://jsonplaceholder

No enviro...

https://jsonplaceholder.typicode.com/albums/1

Save Share

GET https://jsonplaceholder.typicode.com/albums/1

Send

Docs Params Authorization Headers 9 Body Scripts Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 24 Test Results

200 OK 28 ms 1.13 KB

JSON Preview Visualize

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "quidem molestiae enim"
5 }
```





Saul's Workspace

Overview GET https://jsonplaceholder.typicode.com/photos/1

https://jsonplaceholder.typicode.com/photos/1

GET https://jsonplaceholder.typicode.com/photos/1

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 23 Test Results

200 OK • 111 ms • 1.21 KB

```
{
  "albumId": 1,
  "id": 1,
  "title": "accusamus beatae ad facilis cum similique qui sunt",
  "url": "https://via.placeholder.com/600/92c952",
  "thumbnailUrl": "https://via.placeholder.com/150/92c952"
}
```

Saul's Workspace

Overview GET https://jsonplaceholder.typicode.com/todos/1

https://jsonplaceholder.typicode.com/todos/1

GET https://jsonplaceholder.typicode.com/todos/1

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 24 Test Results

200 OK • 28 ms • 1.15 KB

```
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```





HTTP <https://jsonplaceholder.typicode.com/users/1>

GET <https://jsonplaceholder.typicode.com/users/1>

Docs Params Authorization Headers 9 Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers 24 Test Results

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 556",
9     "city": "Gwenborough",
10    "zipcode": "92998-3874",
11    "geo": {
12      "lat": "-37.3159",
13      "lng": "81.1496"
14    }
15 }
```



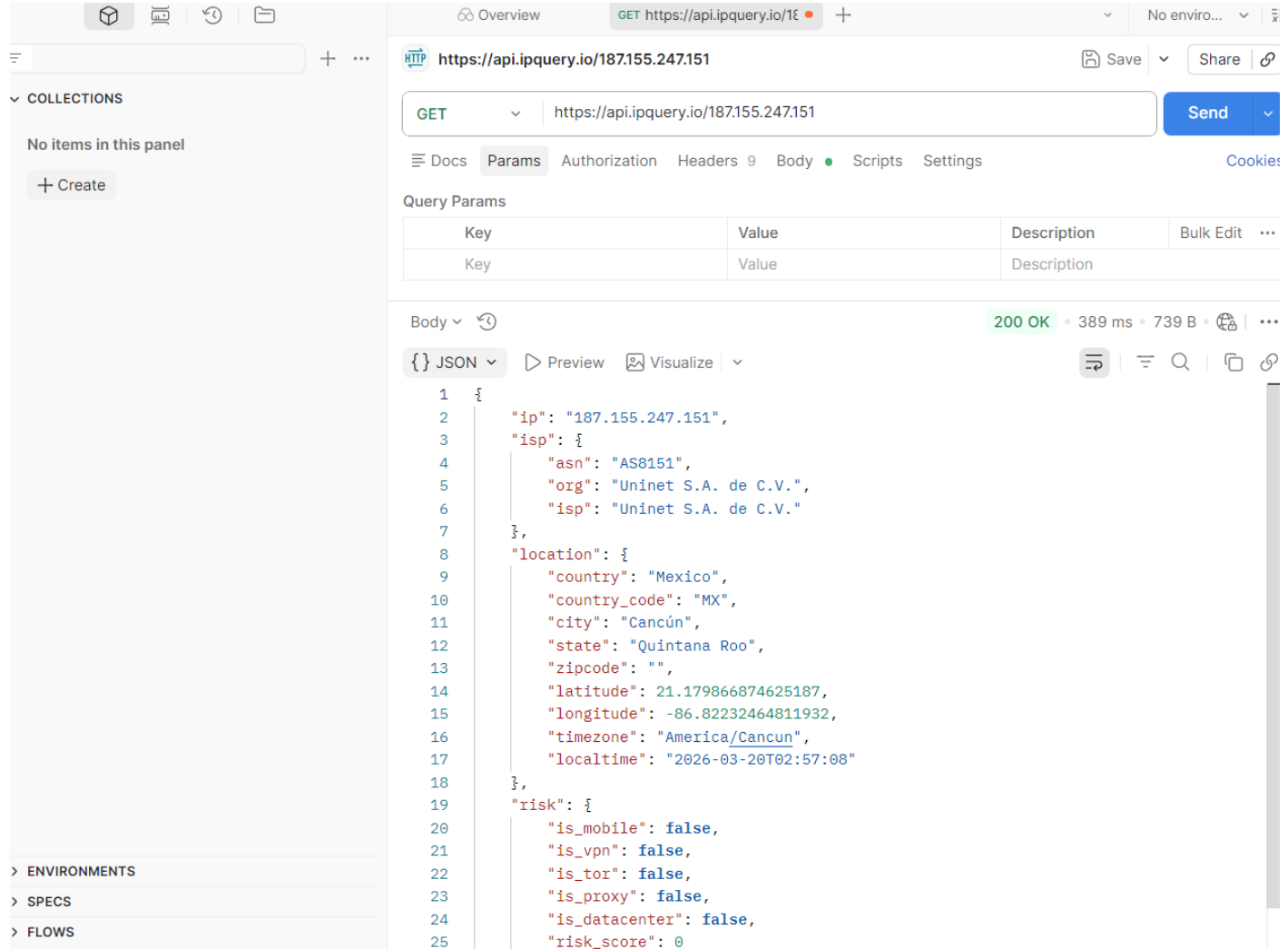
2026
año de
Margarita Maza





Ejercicio 1

1. Extrae la información de tu IP, usando la API: [ipquery.io](https://api.ipquery.io/)
2. Accede a la documentación y realiza las pruebas con postman.
3. Crea un archivo JSON con la información obtenida.



Overview GET https://api.ipquery.io/187.155.247.151

HTTP GET https://api.ipquery.io/187.155.247.151

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body 200 OK • 389 ms • 739 B

```
{
  "ip": "187.155.247.151",
  "isp": {
    "asn": "AS8151",
    "org": "Uninet S.A. de C.V.",
    "isp": "Uninet S.A. de C.V."
  },
  "location": {
    "country": "Mexico",
    "country_code": "MX",
    "city": "Cancún",
    "state": "Quintana Roo",
    "zipcode": "",
    "latitude": 21.179866874625187,
    "longitude": -86.82232464811932,
    "timezone": "America/Cancun",
    "localtime": "2026-03-20T02:57:08"
  },
  "risk": {
    "is_mobile": false,
    "is_vpn": false,
    "is_tor": false,
    "is_proxy": false,
    "is_datacenter": false,
    "risk_score": 0
  }
}
```





```
{
  "ip": "187.155.247.151",
  "isp": {
    "asn": "AS8151",
    "org": "Uninet S.A. de C.V.",
    "isp": "Uninet S.A. de C.V."
  },
  "location": {
    "country": "Mexico",
    "country_code": "MX",
    "city": "Cancún",
    "state": "Quintana Roo",
    "zipcode": "",
    "latitude": 21.179866874625187,
    "longitude": -86.82232464811932,
    "timezone": "America/Cancun",
    "localtime": "2026-03-20T02:57:08"
  },
  "risk": {
    "is_mobile": false,
    "is_vpn": false,
    "is_tor": false,
    "is_proxy": false,
    "is_datacenter": false,
    "risk_score": 0
  }
}
```

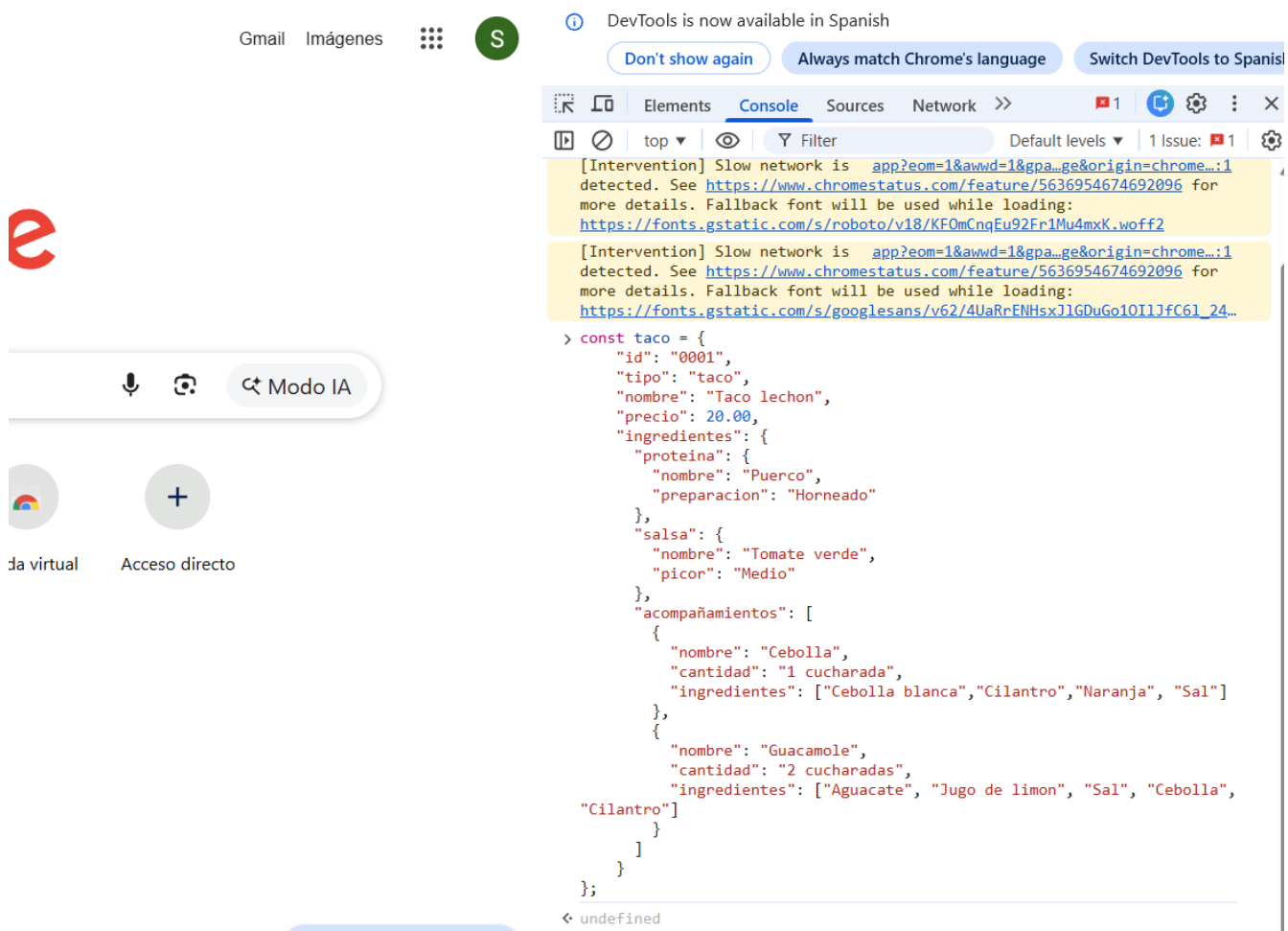




Manipulación de JSON y el DOM

Objetivo: Aprender a navegar por estructuras de datos complejas (objetos anidados y arreglos) utilizando la notación de punto y corchetes.

1. Abre las herramientas de desarrollador en tu navegador (F12 o Ctrl+Shift+I).
2. Ve a la pestaña **Console**.
3. Copia y pega el siguiente objeto **taco** y presiona Enter:



The screenshot shows a web browser with the DevTools Console open. The console displays the following JSON object:

```
const taco = {
  "id": "0001",
  "tipo": "taco",
  "nombre": "Taco lechon",
  "precio": 20.00,
  "ingredientes": {
    "proteina": {
      "nombre": "Puerco",
      "preparacion": "Horneado"
    },
    "salsa": {
      "nombre": "Tomate verde",
      "picon": "Medio"
    },
    "acompañamientos": [
      {
        "nombre": "Cebolla",
        "cantidad": "1 cucharada",
        "ingredientes": ["Cebolla blanca", "Cilantro", "Naranja", "Sal"]
      },
      {
        "nombre": "Guacamole",
        "cantidad": "2 cucharadas",
        "ingredientes": ["Aguacate", "Jugo de limon", "Sal", "Cebolla", "Cilantro"]
      }
    ]
  }
};
```

The console also shows messages about a slow network and font loading.





Escribe el comando `console.log()` necesario para extraer la siguiente información:

- Obtén el nombre de la proteína ("Puerco").

```

< undefined
> taco.ingredientes.proteina.nombre
< 'Puerco'
> |
  
```

- Accede al primer elemento de la lista de ingredientes de la Cebolla ("Cebolla blanca").

```

> console.log(taco.ingredientes.acompañamientos[0].ingredientes[0]);
Cebolla blanca VM228:1
< undefined
  
```

- Accede al último elemento de la lista de ingredientes del Guacamole ("Cilantro").

```

< undefined
> console.log(taco.ingredientes.acompañamientos[1].ingredientes[4]);
Cilantro VM232:1
  
```

Renderizar información dinámica desde un objeto JSON hacia el navegador utilizando JavaScript y manipulación del DOM.

- Crea un archivo llamado `index.html` y utiliza el siguiente código base. Tu tarea es completar la lógica de JavaScript para que la información del taco aparezca en la tarjeta.





TemaDos_APIs > Ejercicio 2.1 > index.html > html > body

```

2  <!DOCTYPE html>
3  <html lang="es">
4  <head>
5    <meta charset="UTF-8">
6    <title>Receta</title>
7    <style>
8      .card { border: 2px solid #2ecc71; border-radius: 10px; padding: 20px; width: 300px; font-family: sans-serif; }
9      .precio { color: #e67e22; font-weight: bold; font-size: 1.2em; }
10     ul { padding-left: 20px; }
11   </style>
12 </head>
13 <body>
14
15   <div id="taco-card" class="card">
16     <h1 id="taco-nombre">Cargando...</h1>
17     <p class="precio">Precio: <span id="taco-precio">0</span></p>
18     <h3>Acompañamientos:</h3>
19     <ul id="lista-acompañamientos"></ul>
20   </div>
21
22   <script>
23     const taco = {
24       "id": "0001",
25       "tipo": "taco",
26       "nombre": "Taco lechon",
27       "precio": 20.00,
28       "ingredientes": {
29         "proteina": {
30           "nombre": "Puerco",
31           "preparacion": "Horneado"
32         },
33         "salsa": {
34           "nombre": "Tomate verde",
35           "picor": "Medio"
36         },
37         "acompañamientos": [
38           {
39             "nombre": "Cebolla",
40             "cantidad": "1 cucharada",
41             "ingredientes": ["Cebolla blanca", "Cilantro", "Naranja", "Sal"]
42           },
43           {
44             "nombre": "Guacamole",
45             "cantidad": "2 cucharadas",
46             "ingredientes": ["Aguacate", "Jugo de limon", "Sal", "Cebolla", "Cilantro"]
47           }
48         ]
49       }
50     };
51
52     // Insertar nombre
53     document.getElementById("taco-nombre").textContent = taco.nombre;
54
55     // Insertar precio
56     document.getElementById("taco-precio").textContent = "$" + taco.precio.toFixed(2);
57
58     // Lista de acompañamientos
59     const lista = document.getElementById("lista-acompañamientos");
60
61     taco.ingredientes["acompañamientos"].forEach(acomp => {
62       const li = document.createElement("li");
63       li.textContent = acomp.nombre + " (" + acomp.cantidad + ")";
64       lista.appendChild(li);
65     });
66   </script>
67
68 </body>
69 </html>

```



2026
año de
Margarita Maza





Receta

Archivo C:/Users/OAT6537/programacion-BackEnd/TemaDos_APIs/Ejercicio%202.1/index.html

Taco lechon

Precio: \$20.00

Acompañamientos:

- Cebolla (1 cucharada)
- Guacamole (2 cucharadas)

JSON (JavaScript Object Notation)

JSON es un formato ligero para el intercambio de datos, basado en la notación de objetos de JavaScript. Se utiliza ampliamente para estructurar datos que se envían a través de Internet.

Los datos en JSON se organizan en pares de clave-valor y pueden incluir arrays y objetos anidados.

La principal diferencia entre un objeto JavaScript y un JSON es que las claves en JSON deben estar entre comillas.

Objeto en JS

```
const objetoJavaScript = {  
  nombre: "Taco de Pollo",  
  ingredientes: {  
    proteina: "Pollo",  
    salsa: "Salsa Verde"  
  }  
};
```

JSON

```
{  
  "nombre": "Taco de Pollo",  
  "ingredientes": {  
    "proteina": "Pollo",  
    "salsa": "Salsa Verde"  
  }  
}
```





```
}  
}
```

1. Para convertir el objeto en una cadena JSON. Crea un archivo .js en tu editor VSC y copia el código del objeto. Seguidamente copia el siguiente código:
2. Observa el resultado.

```
temaDos_API > Ejercicio 2.1 > JS conversion1.js > objetoJavaScript  
1  
2 // Saul Alexander Che Caamal  
3 const objetoJavaScript = {  
4   nombre: "Taco de Pollo",  
5   ingredientes: {  
6     proteina: "Pollo",  
7     salsa: "Salsa Verde"  
8   }  
9 };  
10  
11 //serealizar es convertir a JSON  
12 const jsonString = JSON. stringify(objetoJavaScript);  
13  
14 console. log(jsonString);
```

SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

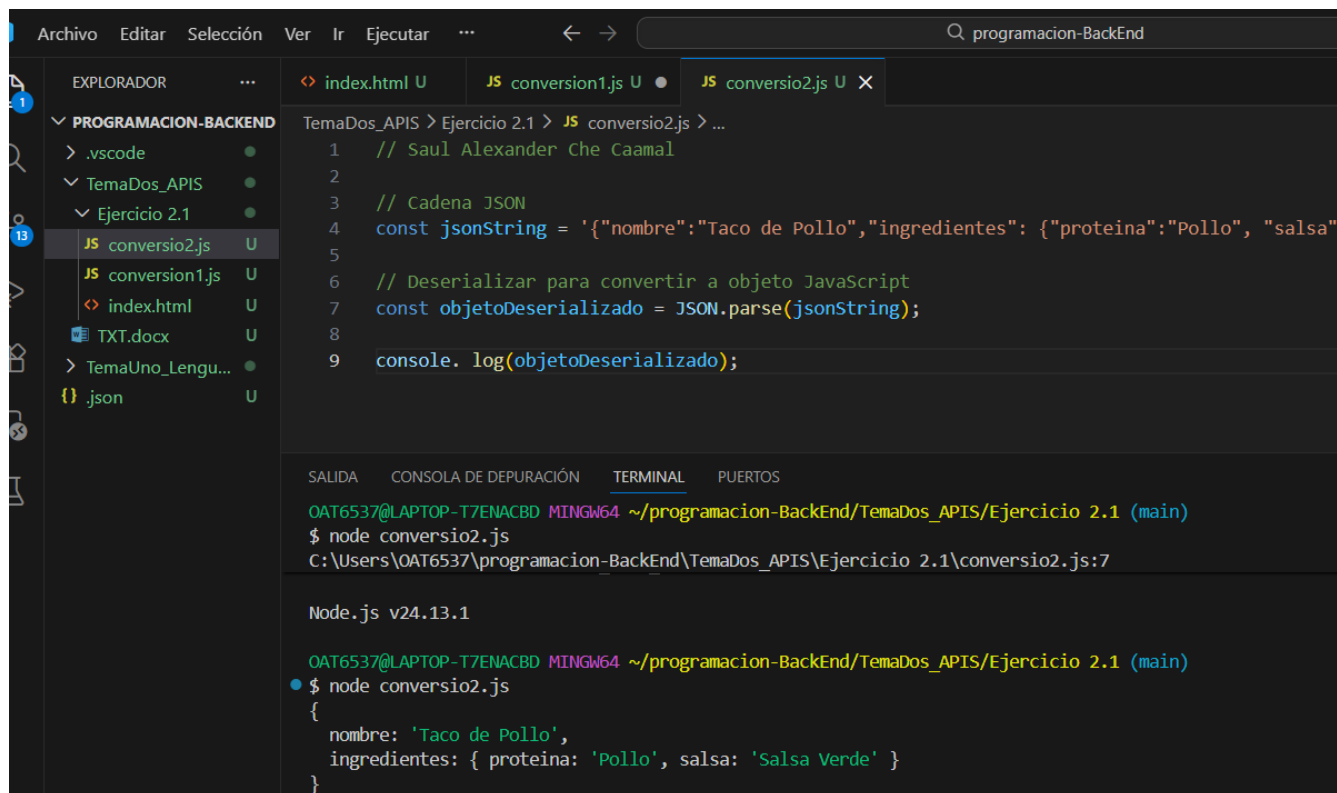
```
at node:internal/main/run_main_module:33:47
```

DAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_API/Ejercicio 2.1 (main)
\$ node conversion1.js
{"nombre":"Taco de Pollo","ingredientes":{"proteina":"Pollo","salsa":"Salsa Verde"}}





3. Para realizar la conversión contraria de un JSON a un objeto JS. Crea otro archivo y copia el siguiente código:



```

1 // Saul Alexander Che Caamal
2
3 // Cadena JSON
4 const jsonString = '{"nombre":"Taco de Pollo","ingredientes":{"proteina":"Pollo", "salsa"
5
6 // Deserializar para convertir a objeto JavaScript
7 const objetoDeserializado = JSON.parse(jsonString);
8
9 console.log(objetoDeserializado);

```

SALIDA CONSOLE DE DEPURACIÓN TERMINAL PUERTOS

OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_API/Ejercicio 2.1 (main)

\$ node conversio2.js

C:\Users\OAT6537\programacion-BackEnd\TemaDos_API\Ejercicio 2.1\conversio2.js:7

Node.js v24.13.1

OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_API/Ejercicio 2.1 (main)

• \$ node conversio2.js

```

{
  nombre: 'Taco de Pollo',
  ingredientes: { proteina: 'Pollo', salsa: 'Salsa Verde' }
}

```

Visualizador de JSON

4. Ejecuta la siguiente petición GET en Postman:

<https://pokeapi.co/api/v2/pokemon/pikachu>





Saul's Workspace

Overview GET https://pokeapi.co/api/v2/pokemon/pikachu

GET https://pokeapi.co/api/v2/pokemon/pikachu

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 27 Test Results

200 OK 300 ms 7.9 KB

{ } JSON Preview Visualize

```

1 {
2   "abilities": [
3     {
4       "ability": {
5         "name": "static",
6         "url": "https://pokeapi.co/api/v2/ability/9/"
7       },
8       "is_hidden": false,
9       "slot": 1
10    },
11    {
12      "ability": {
13        "name": "lightning-rod",
14        "url": "https://pokeapi.co/api/v2/ability/31/"
15      },
16      "is_hidden": true,
17      "slot": 3
18    }
19  ],
20  "base_experience": 112,
21  "cries": {
22    "latest": "https://raw.githubusercontent.com/PokeAPI/cries/main/cries/pokemon/latest/25.ogg",
23    "legacy": "https://raw.githubusercontent.com/PokeAPI/cries/main/cries/pokemon/legacy/25.ogg"
24  },
25  "forms": [
26    {

```

GET htt
Describ
context
AI

5. Copia el resultado y pega en la pestaña Text en el [visualizador de JSON](#)

6. Abre la pestaña Viewer e interpreta la información obtenida.

Viewer Text

JSON

- abilities
- base_experience: 112
- cries
- forms
- game_indices
- height: 4
- held_items
- id: 25
- is_default: true
- location_area_encounters: "https://pokeapi.co/api/v2/pokemon/25/encounters"
- moves
- name: "pikachu"
- order: 35
- past_abilities
- past_stats
- past_types
- species
- sprites
- stats
- types
- weight: 60

Search: GO! Next Previous

Name	Value
abilities	...
base_experience	112
cries	...
forms	...
game_indices	...
height	4
held_items	...
id	25
is_default	true
location_area_encounters	"https://pokeapi.co/api/v2/pokemon/25/encounters"
moves	...
name	"pikachu"
order	35
past_abilities	...
past_stats	...
past_types	...
species	...
sprites	...
stats	...
types	...





7. Crea un archivo json con el nombre **recetaTacos**. Agrega 4 tacos diferentes típicos de la península, con diferente proteína y acompañamientos. Basate en el siguiente ejemplo:

TemaDos_APIs > Ejercicio 2.1 > {} recetaTacos.json > ...

```
1  {
2
3    "id": "0001",
4    "tipo": "taco",
5    "nombre": "Taco de cochinita pibil",
6    "precio": 18.00,
7    "ingredientes": {
8      "proteina": {
9        "nombre": "Cerdo",
10       "preparacion": "Adobado y horneado en hoja de plátano"
11      },
12      "salsa": {
13        "nombre": "Salsa de habanero",
14        "picor": "Alto"
15      }
16    },
17    "acompañamientos": [
18      {
19        "nombre": "Cebolla morada",
20        "cantidad": "1 cucharada",
21        "ingredientes": ["Cebolla morada", "Naranja agria", "Sal"]
22      }
23    ]
24  },
25  {
26    "id": "0002",
27    "tipo": "taco",
28    "nombre": "Taco al pastor",
29    "precio": 20.00,
30    "ingredientes": {
31      "proteina": {
32        "nombre": "Cerdo",
33        "preparacion": "Adobado y cocido en trompo"
34      },
35      "salsa": {
36        "nombre": "Salsa roja",
37        "picor": "Medio"
38      }
39    },
40    "acompañamientos": [
41      {
42        "nombre": "Piña con cebolla",
43        "cantidad": "1 porción",
44        "ingredientes": ["Piña", "Cebolla", "Cilantro", "Limón"]
45      }
46    ]
47  },
48  {
49    "id": "0003",
50    "tipo": "taco",
51    "nombre": "Taco de res",
52    "precio": 15.00,
53    "ingredientes": {
54      "proteina": {
55        "nombre": "Res",
56        "preparacion": "Asado y cortado en tiras"
57      },
58      "salsa": {
59        "nombre": "Salsa verde",
60        "picor": "Medio"
61      }
62    },
63    "acompañamientos": [
64      {
65        "nombre": "Cebolla blanca",
66        "cantidad": "1 cucharada",
67        "ingredientes": ["Cebolla blanca", "Cilantro", "Limón"]
68      }
69    ]
70  },
71  {
72    "id": "0004",
73    "tipo": "taco",
74    "nombre": "Taco de pollo",
75    "precio": 12.00,
76    "ingredientes": {
77      "proteina": {
78        "nombre": "Pollo",
79        "preparacion": "Asado y desmenuzado"
80      },
81      "salsa": {
82        "nombre": "Salsa blanca",
83        "picor": "Medio"
84      }
85    },
86    "acompañamientos": [
87      {
88        "nombre": "Cebolla blanca",
89        "cantidad": "1 cucharada",
90        "ingredientes": ["Cebolla blanca", "Cilantro", "Limón"]
91      }
92    ]
93  }
94  ]
```



2026
año de
Margarita
Maza





```
49  {
50    "id": "0003",
51    "tipo": "taco",
52    "nombre": "Taco de asada",
53    "precio": 22.00,
54    "ingredientes": {
55      "proteina": {
56        "nombre": "Res",
57        "preparacion": "Asada a la parrilla"
58      },
59      "salsa": {
60        "nombre": "Salsa verde",
61        "picor": "Medio"
62      }
63    },
64    "acompañamientos": [
65      {
66        "nombre": "Guacamole",
67        "cantidad": "2 cucharadas",
68        "ingredientes": ["Aguacate", "Limón", "Sal", "Cilantro"]
69      }
70    ],
71  },
72  {
73    "id": "0004",
74    "tipo": "taco",
75    "nombre": "Taco de pollo pibil",
76    "precio": 17.00,
77    "ingredientes": {
78      "proteina": {
79        "nombre": "Pollo",
80        "preparacion": "Adobado y horneado"
81      },
82      "salsa": {
83        "nombre": "Salsa de habanero",
84        "picor": "Alto"
85      }
86    },
87    "acompañamientos": [
88      {
89        "nombre": "Cebolla morada",
90        "cantidad": "1 cucharada",
91        "ingredientes": ["Cebolla morada", "Naranja agria", "Sal"]
92      }
93    ]
94  }
```



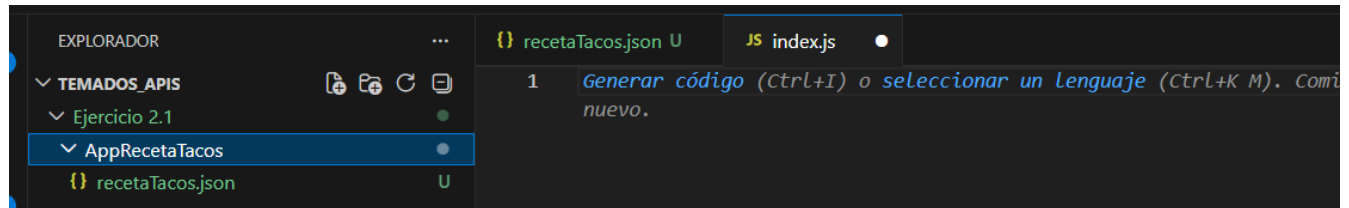
2026
año de
Margarita Maza



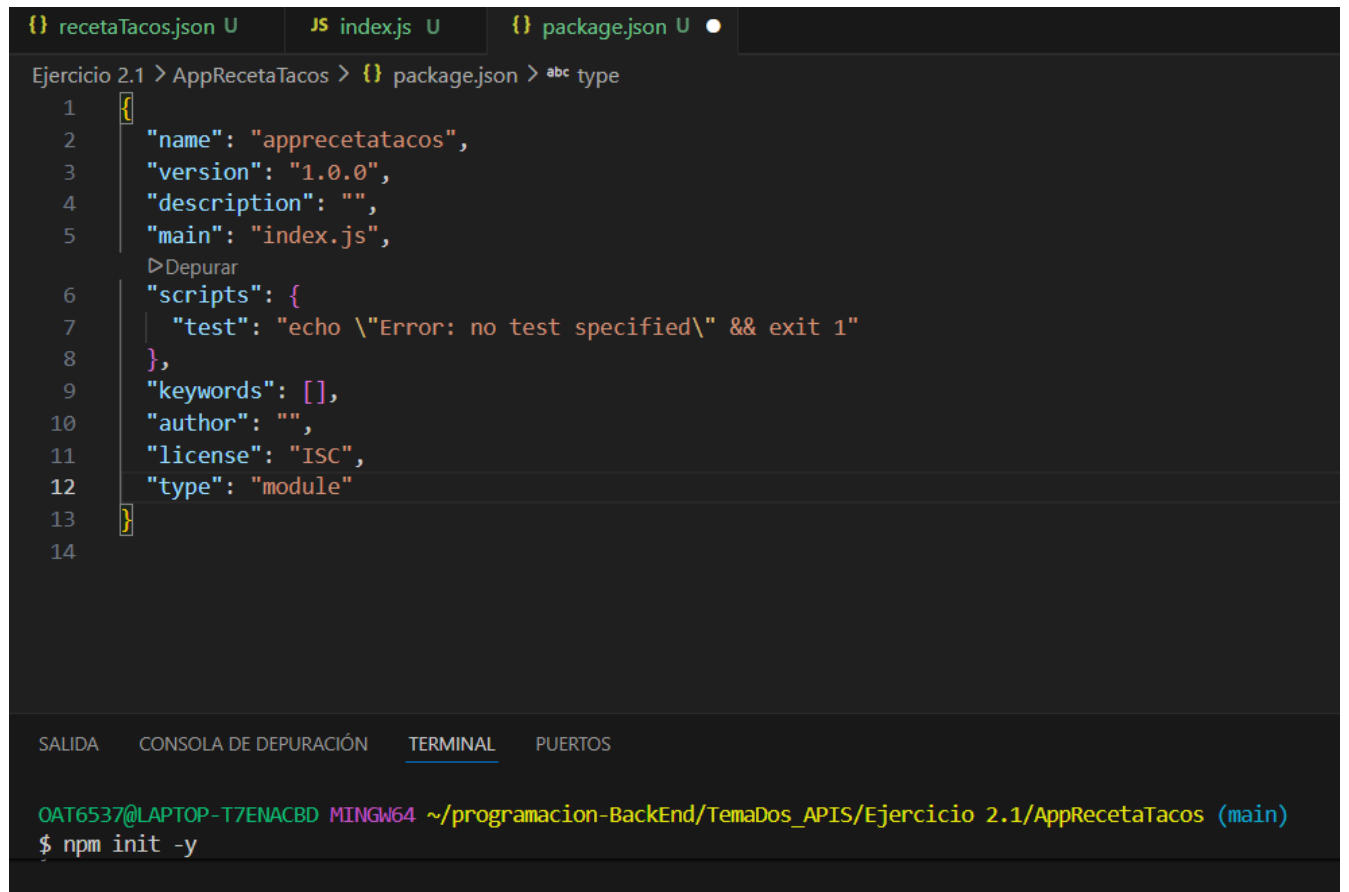


Aplicación usando datos JSON y API Fetch.

1. Crea el proyecto con el nombre **AppRecetaTacos**
2. Crea el archivo **index.js**
3. Agregar el archivo **recetaTacos.json**



4. Inicializa npm
5. Agrega el tipo **module** como sistema de módulos en el archivo configuración.





6. Instala el framework **express** y el middleware **body-parser**

```
OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_APIs/Ejercicio 2.1/AppRecetaTacos (main)
$ npm install express body-parser

added 65 packages, and audited 66 packages in 6s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

7. Crea el boilerplate de la aplicación:

1. Importar las dependencias instaladas.
2. Crear la instancia de express
3. Configurar el puerto
4. Configurar el método para iniciar el servidor.
5. Probar funcionalidad

8. Crear el directorio **public** con los archivos **index.html** y **estilo.css**

```
index.html U x # estilo.css U
Ejercicio 2.1 > AppRecetaTacos > public > index.html
1 |Generar código (Ctrl+I) o seleccionar un lenguaje (Ctrl+K M). Comience a escribir para descartar o no mostrar esto nuevo.

SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_APIs/Ejercicio 2.1/AppRecetaTacos (main)
$ cd p
package.json  package-lock.json  public/

OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_APIs/Ejercicio 2.1/AppRecetaTacos (main) ...

OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_APIs/Ejercicio 2.1/AppRecetaTacos/public (main)
$ code index.html

OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_APIs/Ejercicio 2.1/AppRecetaTacos/public (main)
$ code estilo.css

OAT6537@LAPTOP-T7ENACBD MINGW64 ~/programacion-BackEnd/TemaDos_APIs/Ejercicio 2.1/AppRecetaTacos/public (main)
$
```





9. Crea el boilerplate para un documento html.

1. DOCTYPE y HTML.
2. Cabecera <head>, para definir título para la página TacosMx
3. Enlazar con la [hoja de estilos](#) que se utilizará para la página.
4. Integrar al cuerpo <body>, el título principal (<h1>Tacoschidos 🌮 </h1>).
5. Incluye en el body un formulario con varios botones, cada botón representa la proteína de cada tipo de taco. Finalmente deben haber 4 botones. Todos los botones tendrán un atributo data-type que se utiliza para identificar el tipo de taco seleccionado. La estructura es la siguiente:

```
<form id="Form" class="buttons">
  <button type="button" data-type="Puerco">🐷</button>
</form>
```

f. Después del formulario, integrar el siguiente elemento contenedor:

```
<div id="Container"></div>
```

g. Para obtener los datos del backend y renderizarlos en el navegador utilizando el DOM de JS, se integrará el [script](#) utilizando la API Fetch.

10. Desarrolla el backend:

- a. Crea una constante **recetaJSON** y asigna entre backtips la cadena JSON generada en tu archivo recetaTacos.json.
- b. Deserializa (convierte a objeto JS) la cadena JSON y asigna a una constante **recetasTacos**.
- c. Utiliza el middleware **express.static** para servir los archivos index.html y estilo.css desde la carpeta public:

```
nombre_servidor.use(express.static("public"));
```

d. Utiliza el middleware **bodyParser.json()** para acceder a los datos enviados en el cuerpo de una solicitud.

e. Crea el handler **get** para obtener la receta de un taco en específico. El endpoint es **/receta/:type**. El objetivo del siguiente código es encontrar una receta de taco que coincida con el tipo de proteína especificado en la solicitud. El contenido del handler debe ser el siguiente:

```
const elegirTaco = recetasTacos.find(r => r.ingredientes.proteina.nombre.toLowerCase() === req.params.type.toLowerCase());

res.json(elegirTaco || { error: "Receta no encontrada" });
```

req.params.type() proviene de la ruta dinámica en la URL, como /receta/:type, donde :type es un valor proporcionado por el cliente ("pollo" o "puerco").

f. Prueba la funcionalidad. El resultado deberá ser similar a la imagen.





```
Ejercicio 2.1 > AppRecetaTacos > public > index.html > html > body > div#Container
> Cochinita Aa ab .* ? de 1 ↑ ↓ ≡ ×

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>TacosMx</title>
7   <link rel="stylesheet" href="estilo.css">
8 </head>
9 <body>
10   <h1>Tacoschidos 🌮</h1>
11
12   <form id="Form" class="buttons">
13     <button type="button" data-type="Cochinita">🐷</button>
14     <button type="button" data-type="Pastor">🐔</button>
15     <button type="button" data-type="Res">🐝</button>
16     <button type="button" data-type="Pollo">🐔</button>
17   </form>
18
19   <div id="Container"></div>
20   <script>
21     document.querySelectorAll("#Form button").forEach(button => {
22       button.addEventListener("click", async (event) => {
23         const type = event.target.getAttribute("data-type").toLowerCase();
24
25         const response = await fetch(`/receta/${type}`);
26         const receta = await response.json();
27
28         const contenedor = document.getElementById("Container");
29         if (receta.error) {
30           contenedor.innerHTML = "<h2>Receta no encontrada</h2>"; //si no se recupera el contenido HTML de un elemento.
31         } else {
32           contenedor.innerHTML = `
33             <h2>${receta.nombre}</h2>
34           `
35         }
36       });
37     });
38   </script>
39 </body>
40 </html>
```



2026
año de
Margarita Maza





```

JS index.js U  < index.html U  # estilo.css U x  .gitignore U  {} recetaTacos.json U
Ejercicio 2.1 > AppRecetaTacos > public > # estilo.css > #ingredientesLista li
1  html {
2    height: 100%;
3  }
4
5  body {
6    font-family: Arial, sans-serif;
7    margin: 20px;
8    padding: 0;
9    text-align: center;
10 }
11
12 h1 {
13   display: inline;
14   font-size: 5rem;
15   text-align: center;
16   color: #e26e5c;
17   background-color: white;
18 }
19
20 h2 {
21   color: #195784;
22 }
23
24 .buttons {
25   display: flex;
26   justify-content: center;
27   margin: 20px;
28 }
29
30 button {
31   font-size: 5rem;
32 }
33
34 .buttons button {
35   display: flex;

```

